JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class CollationKey

java.lang.Object
    java.text.CollationKey

All Implemented Interfaces:

Comparable<CollationKey>

```
public abstract class CollationKey
extends Object
implements Comparable<CollationKey>
```

A `CollationKey` represents a `String` under the rules of a specific `Collator` object. Comparing two `CollationKeys` returns the relative order of the `Strings` they represent. Using `CollationKeys` to compare `Strings` is generally faster than using `Collator.compare`. Thus, when the `Strings` must be compared multiple times, for example when sorting a list of `Strings`. It's more efficient to use `CollationKeys`.

You can not create `CollationKeys` directly. Rather, generate them by calling `Collator.getCollationKey`. You can only compare `CollationKeys` generated from the same `Collator` object.

Generating a `CollationKey` for a `String` involves examining the entire `String` and converting it to series of bits that can be compared bitwise. This allows fast comparisons once the keys are generated. The cost of generating keys is recouped in faster comparisons when `Strings` need to be compared many times. On the other hand, the result of a comparison is often determined by the first couple of characters of each `String`. `Collator.compare` examines only as many characters as it needs which allows it to be faster when doing single comparisons.

The following example shows how `CollationKeys` might be used to sort a list of `Strings`.

```
// Create an array of CollationKeys for the Strings to be sorted.
Collator myCollator = Collator.getInstance();
CollationKey[] keys = new CollationKey[3];
keys[0] = myCollator.getCollationKey("Tom");
keys[1] = myCollator.getCollationKey("Dick");
keys[2] = myCollator.getCollationKey("Harry");
sort(keys);

//...

// Inside body of sort routine, compare keys this way
```

```
if (keys[i].compareTo(keys[j]) > 0)
   // swap keys[i] and keys[j]

//...

// Finally, when we've returned from sort.
System.out.println(keys[0].getSourceString());
System.out.println(keys[1].getSourceString());
System.out.println(keys[2].getSourceString());
```

Since:

1.1

See Also:

Collator, RuleBasedCollator

Constructor Summary

Constructors

Modifier	Constructor	Description
protected	**CollationKey**(String source)	CollationKey constructor.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
abstract int	**compareTo**(CollationKey target)	Compare this CollationKey to the target CollationKey.
String	**getSourceString**()	Returns the String that this CollationKey represents.
abstract byte[]	**toByteArray**()	Converts the CollationKey to a sequence of bits.

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Details

CollationKey

```
protected CollationKey(String source)
```

CollationKey constructor.

Parameters:

source - the source string

Throws:

NullPointerException - if source is null

Since:

1.6

Method Details

compareTo

```
public abstract int compareTo(CollationKey target)
```

Compare this CollationKey to the target CollationKey. The collation rules of the Collator object which created these keys are applied. **Note:** CollationKeys created by different Collators can not be compared.

Specified by:

compareTo in interface Comparable<CollationKey>

Parameters:

target - target CollationKey

Returns:

Returns an integer value. Value is less than zero if this is less than target, value is zero if this and target are equal and value is greater than zero if this is greater than target.

See Also:

Collator.compare(String, String)

getSourceString

```
public String getSourceString()
```

Returns the String that this CollationKey represents.

Returns:

the source string of this CollationKey

toByteArray

```
public abstract byte[] toByteArray()
```

Converts the CollationKey to a sequence of bits. If two CollationKeys could be legitimately compared, then one could compare the byte arrays for each of those keys to obtain the same result. Byte arrays are organized most significant byte first.

Returns:

a byte array representation of the CollationKey

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.